RECURSION

Key idea: divide problem into smaller similar subproblem. **Base case**: smallest, immediate solution (no recursive call). **Recursive case**: calls itself on simpler input approaching base case. Each call has its own local variables. **Largest**: compare first elem to largest of rest; base=size 1. **Reverse**: remove first, reverse rest, append first; base=size 1. **Member-of**: `x==head || member(x,rest)`; base=size 0 (false). **Multiply** `i*j`: `i + multiply(i,j-1)`; base `j==0`→0.

Towers of Hanoi (n disks, src→dst via tmp):

- `solve(n-1,src,dst,tmp)` move top n-1 to tmp
- `solve(1,src,tmp,dst)` move largest to dst (base)
- `solve(n-1,tmp,src,dst)` move n-1 from tmp to dst

Very hard without recursion. 2^n-1 moves total.

Fibonacci Disaster: `f(n)=f(n-2)+f(n-1)`, base `f(0)=0,f(1)=1`. Exponential calls – use iteration instead. Recursion is powerful but use appropriately.

Quicksort Partition: pick pivot (middle), swap to right, move `i` right while `data[i]`, move `j` left while `data[j]>pivot`, if `i` swap `data[i],data[j]`, repeat until `j` crosses `i`, swap pivot back. Pivot at correct position. Recursively sort left/right subarrays. Base: size 0/1 return; size 2 swap if needed. Most elegant algorithm (1961).

Dynamic Backtracking: try a solution path recursively; if dead end, backtrack and try another. Decision tree: stages, nodes=steps, paths between stages. Base=step with no solution paths.

N-Queens: solve column by column L→R. `is\_safe\_position(row,col)`: check attacks from left. `find\_solutions()`: recurse on columns to right, then backtrack to next safe square. Base=past last column→solution found. 92 solutions for 8 queens.

Sudoku Backtracking: brute force, try 1-9 at each cell L→R, top→bottom. Recursively check ahead; if no solution backtrack. Base=past last cell→solved.

MULTITHREADING

Thread: path computer takes through code. Multithreading = multiple simultaneous paths. Each thread has own stack & registers; share code, data, resources. **Concurrency**: one CPU switches tasks. **Parallelism**: multiple CPUs truly simultaneous.

Shared resource: memory, printer, output stream.

Critical region: code that accesses shared resource. **Mutual exclusion**: only one thread in critical region at a time.

mutex: enforces mutual exclusion. `mutex m; m.lock(); /*crit*/ m.unlock();`. Thread blocks if mutex already locked. Must unlock before exiting critical region.

lock_guard: RAII wrapper – auto-locks on construction, auto-unlocks on destruction (scope exit). `lock\_guard lk(m);`. Prevents forgetting to unlock (deadlock risk).

Context switching: runtime switches among threads. `this\_thread::yield()` explicitly relinquishes control. `this\_thread::sleep\_for(chrono::seconds(n))`.

Reader-Writer Problem: multiple readers OK simultaneously; one writer excludes all. Use `shared\_mutex`. `unique\_lock`: one thread only (writer). `shared\_lock`: multiple threads (readers). Both auto-unlock on scope exit.

atomic< T>: thread-safe ops without mutex. `atomic cnt; cnt.store(n); cnt.load(); -cnt;`

condition_variable: synchronization beyond mutexes. `cv.wait(lock)`: unlocks, sleeps, re-locks when notified. `cv.notify\_all()`: wake all waiting threads. Pattern: `while(condition) cv.wait(lock);` (loop guards spurious wakeups).

Producer-Consumer: producers enter data to shared queue (capacity-limited); consumers remove. Queue full→producers wait on `producer\_cv`; queue empty→consumers wait on `consumer\_cv`. Producer notifies consumers when queue becomes non-empty; consumer notifies producers when queue becomes non-full.

Dining Philosophers: 5 philosophers, 5 chopsticks (1 between each). Need both adjacent chopsticks to eat. Must pick up both at once (not hold-and-wait). Loop: think→wait→fill bowl→eat.

Deadlock: all threads blocked, app hangs. Avoid by consistent lock ordering, using `lock\_guard`. Performance: more threads \ne faster; context switch costs, mutex overhead matter. **Spawn**: `thread t(func, args); t.join();`

auto: type inference from initializer. `auto x = expr;` compiler deduces type. **decltype(x)**: returns type of variable `x`. Use: `auto t = steady\_clock::now(); decltype(t) end;`

WELL-DESIGNED SOFTWARE

• Meets requirements (does what supposed to) • Reliable (fewer bugs, no surprises, no hidden perf issues) • Flexible & scalable (easy to add features, less complexity) • Maintainable (future devs can understand) • Uses good design techniques & patterns • Saves time/cost overall. Good design = iterative dev (design-code-test).

OOP DESIGN

Encapsulation: combine data+behavior in class; public/private; contains/isolates changes (prevents leaks). Object **state** = values of member vars; **state change** = values change.

Abstraction: ignore irrelevant details; multiple levels; reduces complexity.

Inheritance: subclass from superclass; inherits state+behavior; can add/override. Reduces code complexity.

Polymorphism: pointer to superclass calls correct subclass method at runtime. `virtual` keyword enables. Increases flexibility. `override` keyword: compiler checks signature match.

dynamic_cast< T*>(ptr): type check+convert; returns `nullptr` if wrong type. **Abstract class**: ≥1 pure virtual fn (`=0`); cannot instantiate directly.

Interface class: only pure virtual fns, no member vars. Subclasses must implement all. Use dashed arrow in UML (realization).

virtual destructor: required if class has subclasses. **Function signature**: name + param count/types/order (not return type, not param names).

Override vs Overload: Override=subclass same signature (polymorphism). Overload=same name different signature same scope.

Liskov Substitution: subclass object substitutable for superclass without errors. Violation: `CircularBuffer : vector` allows invalid `at(),erase()` – fix: has-a not is-a.

Code to Interface: declare as superclass ptr, assign subclass. `Shape *s = new Rectangle(); s = new Circle();` Relies on polymorphism. Hardcoding: `Rectangle *r = new Rectangle();` – inflexible.

DESIGN SMELLS

• **Leaking changes**: change in class A forces change in B (tight coupling) • **God class**: too many responsibilities (AutomobileApp) • **Hardcoded concrete**: `Cat *pet = new Cat();` inflexible • **Least surprise violation**: misleading names, hidden perf issues, off-by-one (month array) • **DRY violation**: repeated code in multiple places

DESIGN PRINCIPLES

SRP (Single Responsibility): class has one primary responsibility; cohesive; easy to use+maintain.

Encapsulate What Varies: isolate code that can change so changes don't leak to other code. Put varying code in its own class.

Delegation: move work from requester class to more suitable delegate class. Requester has no dependency on how delegate works. Increases cohesion.

Principle of Least Knowledge (Law of Demeter): classes have minimal dependencies on each other's implementation. Loosely coupled. Rules: `afunc()` may call `bfunc()` on B if: (1) B is member var of A; (2) B passed as param to afunc; (3) B instantiated by afunc. May NOT call on B returned by another object's method (`a.getB().getC()` = smell). Return copies not pointers (immutability).

OCP (Open-Closed): closed to modification (stable), open for extension (subclasses). Use private members in superclass.

Code to Interface: write code using superclass/interface, not specific subclasses. Runtime polymorphism.

DRY (Don't Repeat Yourself): no duplicate code; share one copy as function or class.

Defensive Programming: runtime checks of param values; create only valid objects; `assert()` invariants.

Programming by Contract: **Precondition**: must be true when function called (caller's responsibility). **Postcondition**: must be true when function returns (function's responsibility). **Class invariant**: condition true after construction and after every mutation. `assert(class\_invariant());` Use `assert()` for pre/post conditions.

Favor Composition over Inheritance: "has-a" often better than "is-a". Toys: `Toy` has `PlayAction*` and `Sound*` – strategies composable. Factory function: `static Toy* make(Kind k)` creates/returns correct subclass.

Lazy Evaluation: compute value only when needed.

Refactoring: redesign to improve without changing functionality.

REQUIREMENTS

Functional req: what system shall do or allow users to do. Use "shall"/"must". **Nonfunctional req**: usability, reliability, performance, supportability constraints. Just as important! Requirements must be: **Clear** (jargon-free), **Consistent** (no contradictions), **Correct**, **Complete** (no gaps), **Realistic**, **Verifiable** (testable), **Traceable** (each req ↔ feature). Use strong auxiliary verbs "shall"/"must"; "should" = wish list. Sources: interviews, observation, stated+implied requirements. Iterative: show prototypes, requirements evolve.

I18N (Internationalization, 18 letters): design app to support multiple locales. **L10N** (Localization, 10 letters): adapt to specific locale (language, date format, monetary, encoding). Encapsulate locale-varying parts.

V&V: **Validate** = building right application (customer confirms requirements) – customers validate. **Verify** = building application right (developers test implementation) – developers verify.